and more reliable. When applications have messy or inconsistent HTML code, the elements can be hard to identify, which results in more complicated and more brittle selector logic.

The technology stack you choose can have a major effect on testability. Frameworks that limit the control you have over the rendered HTML are a major source of difficulty. In Java web application development, for example, some frameworks automatically generate element identifiers for their own use, making it difficult to use the simplest and fastest of the WebDriver selectors.[3] Applications that use plugin technologies such as Flash and Silverlight, which are opaque to testing tools like WebDriver, also make testing very difficult.

8.3 Using page objects to make your tests cleaner

Up until now we've explored the WebDriver API using simple code samples. These examples work well to illustrate the various WebDriver methods, but you wouldn't want to write code like this for real-world automated tests. For example, to log on to the Frequent Flyer website, you used the following code:

```
driver.get("http://localhost:8080/#/welcome");
driver.findElement(By.name("email")).sendKeys("jane.smith@acme.com");
driver.findElement(By.name("password")).sendKeys("s3cr3t");
driver.findElement(By.id("signin")).click();
```

This code wouldn't scale well. You'd need to duplicate the same or similar lines for every scenario involving a user logging on, and any change to this logic would need to be updated at every place that it's used.

Another problem is that you're mixing selector logic (`By.name()` and so on) with test data (`jane.smith@acme.com` and so forth), which prevents you from reusing the locators in other tests.

A better approach would be to refactor the selector logic into one place so that it can be reused across multiple tests. You could write a class along the following lines to do this:

```
public class LoginPage {

    private final WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void open() {
        driver.get("http://localhost:8080/#/welcome");
    }

    public void signinWithCredentials(String userEmail,
                                      String userPassword) {
```

1. **Declare a WebDriver instance that the page object can use to interact with the browser.**
2. **Pass in the WebDriver instance.**
3. **Open the login page.**
4. **Enter user credentials and sign in to the application.**

[3] Many JSF-based frameworks fall into this category.